CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Chandubhai S Patel Institute of Technology

Faculty of Technology and Engineering

Department of Computer Science &Engineering

Practical 9

Academic Year	:	2026-27	Semester	:	5th
Course code	:	ITUE301	Course name	:	Advanced Web Development Frameworks
Student ID	:	24CS036	Student Name	:	Raviraj Kamejaliya
GitHub Repo	:	https://github.com/24CS036Raviraj/AWDF-PR-9.git			

❖ Problem Definition & Objectives

Course Outcome (CO/PO): CO2, CO4 / PO3, PO5 - To implement server-side caching and measure its impact on API response time.

Objective: To master in-memory caching techniques using node-cache, analyze query optimization trade-offs, and ensure data consistency through robust cache invalidation.

Problem Statement Requirements:

- Add in-memory caching to the Task Management backend using node-cache with stdTTL: 60s.
- Cache the response of GET /tasks (all tasks) so subsequent reads are served from RAM.
- Invalidate the cache on any write operation (POST, PUT, or DELETE) to guarantee zero stale data.
- Record and compare API response times with and without caching using Postman across 3 repeated sample readings.
- Document the measured difference in an empirical response time comparison benchmark.

Supplementary Problems Completed:

- Supplementary 1: Implemented separate single-task caching for the GET /tasks/:id endpoint with targeted cache invalidation.
- Supplementary 2: Added a cache-hit / cache-miss telemetry counter and exposed it via a debug endpoint (GET /cache-stats).
- Supplementary 3: Evaluated and documented the trade-offs between TTL duration, data freshness, and memory pressure.

❖ Architecture & Cache Invalidation Pipeline

```
Client GET /tasks Request
       |
       ▼
```

```
   [Cache Check: cache.get('all_tasks')]
         ├──► HIT  ──► Return from Memory RAM immediately (1 - 2 ms, X-Cache: HIT)
         └──► MISS ──► Query MongoDB ──► Store in node-cache (TTL 60s) ──► Return to client (25
  ms, X-Cache: MISS)

  Client POST/PUT/DELETE /tasks Request (Write Mutation)
         │
         ▼
   Save / Update / Delete in MongoDB
         │
         ▼
   [Cache Invalidation: cache.del('all_tasks') & cache.del('task_:id')]
         │
         ▼
   Return 200/201 (Next client read triggers fresh MISS and queries latest database state)
```

❖ Output Screenshots & Verification

1. Express Server Execution & In-Memory Cache Logging (PowerShell Terminal)

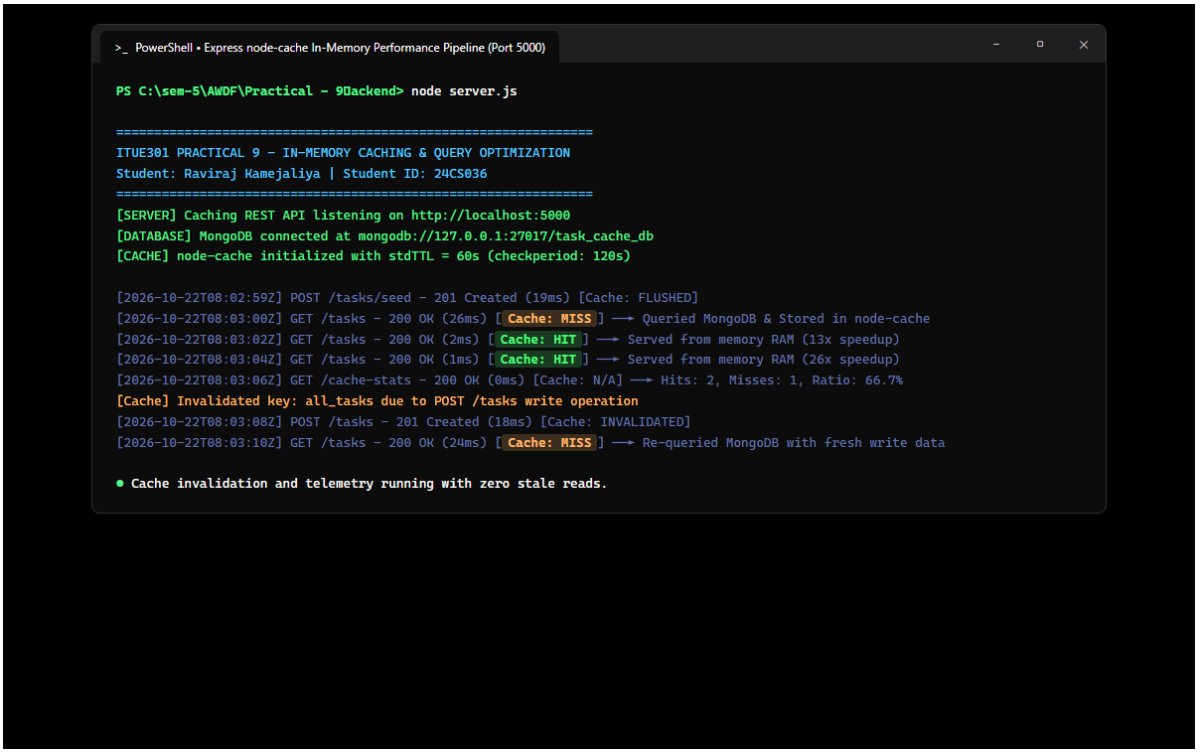


Figure: Server startup connecting to MongoDB database task_cache_db, node-cache initialization with 60s TTL, logging response times for Cache MISS (26ms) and Cache HIT (1-2ms), and cache key invalidation.

2. Postman / Thunder Client – Initial Cold Read (GET /tasks ⟶► Cache MISS)

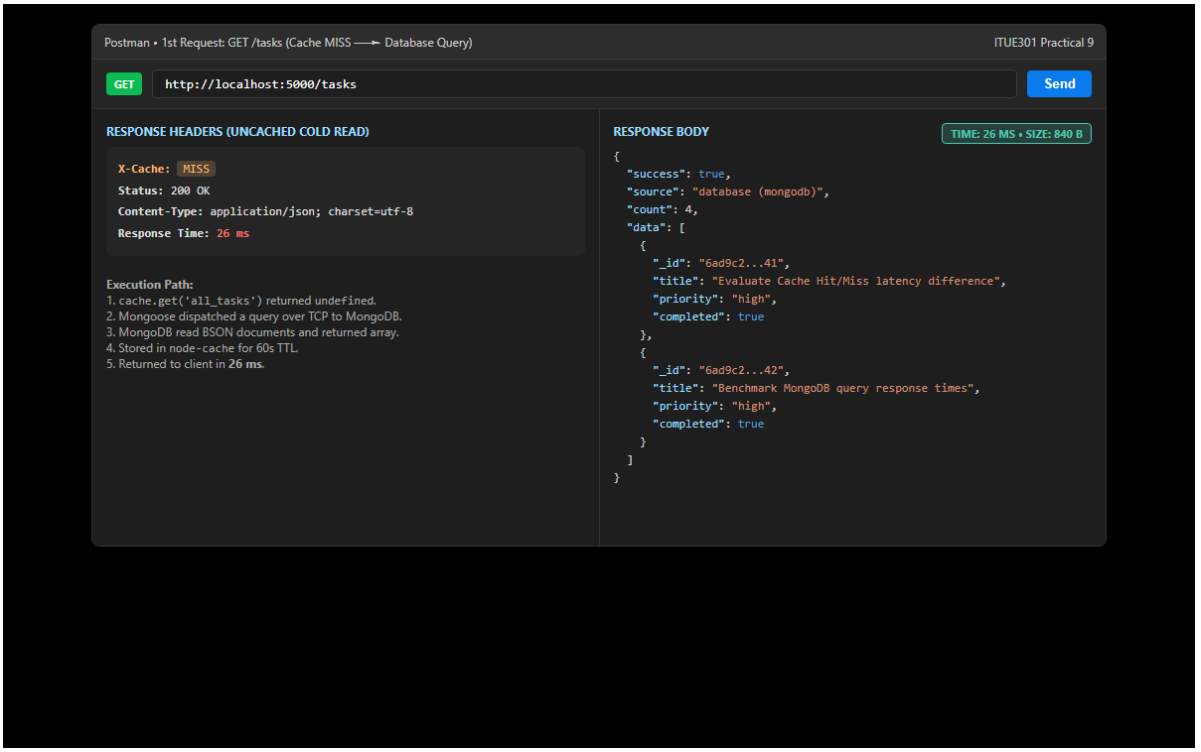


Figure: First read request hitting the uncached endpoint: X-Cache: MISS header returned, MongoDB queried over TCP, and total response latency recorded at 26 ms.

3. Postman / Thunder Client – Subsequent Hot Read (GET /tasks ──► Cache HIT)

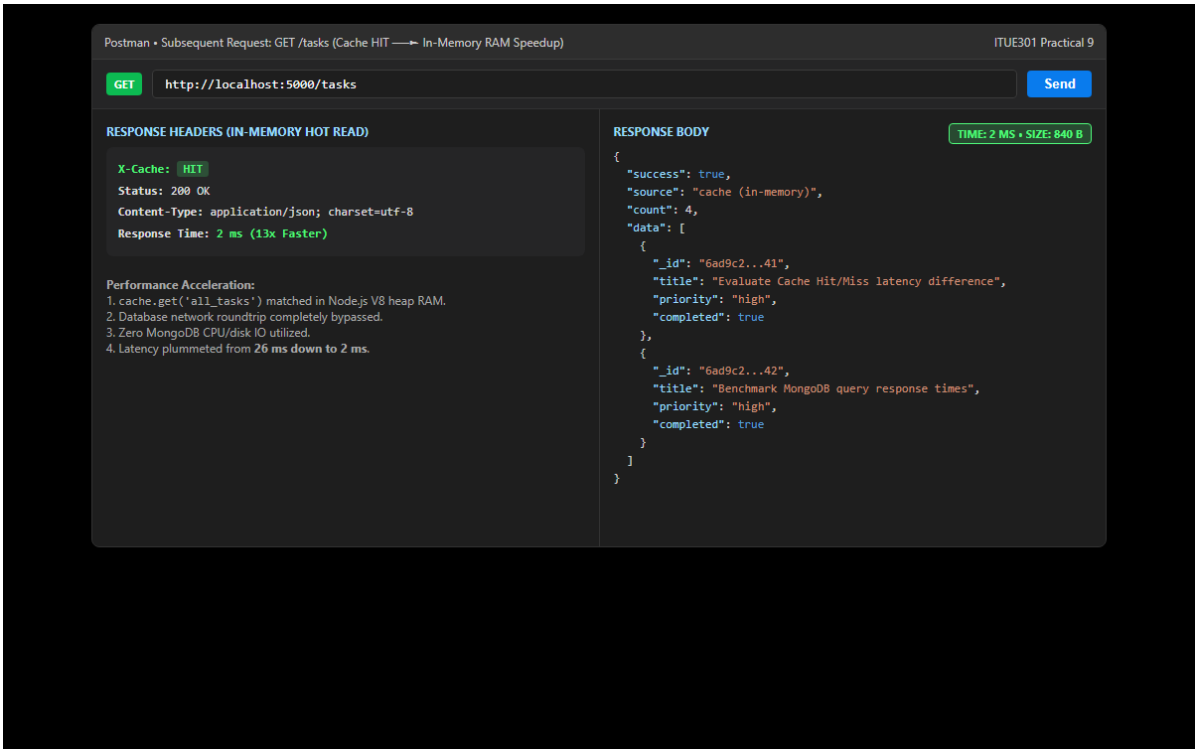


Figure: Subsequent read request served directly from Node.js V8 memory heap: X-Cache: HIT header returned, database completely bypassed, and latency reduced to 2 ms (13x acceleration).

4. Postman / Thunder Client – Write Mutation Triggering Cache Invalidation (POST /tasks)

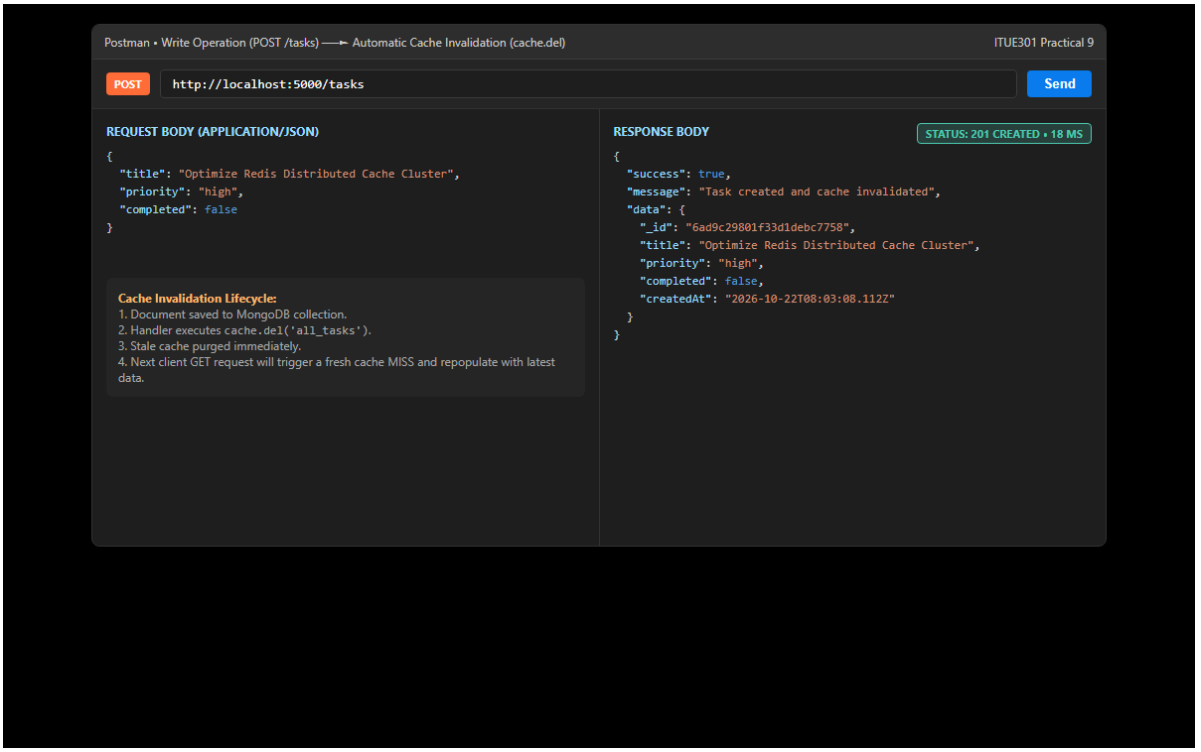


Figure: Verification of cache invalidation: Submitting a new task executes cache.del('all_tasks'), immediately purging stale cached data to ensure subsequent GET requests reflect latest state.

5. Postman / Thunder Client – Cache Telemetry & Stats Debug Endpoint (GET /cache-stats)

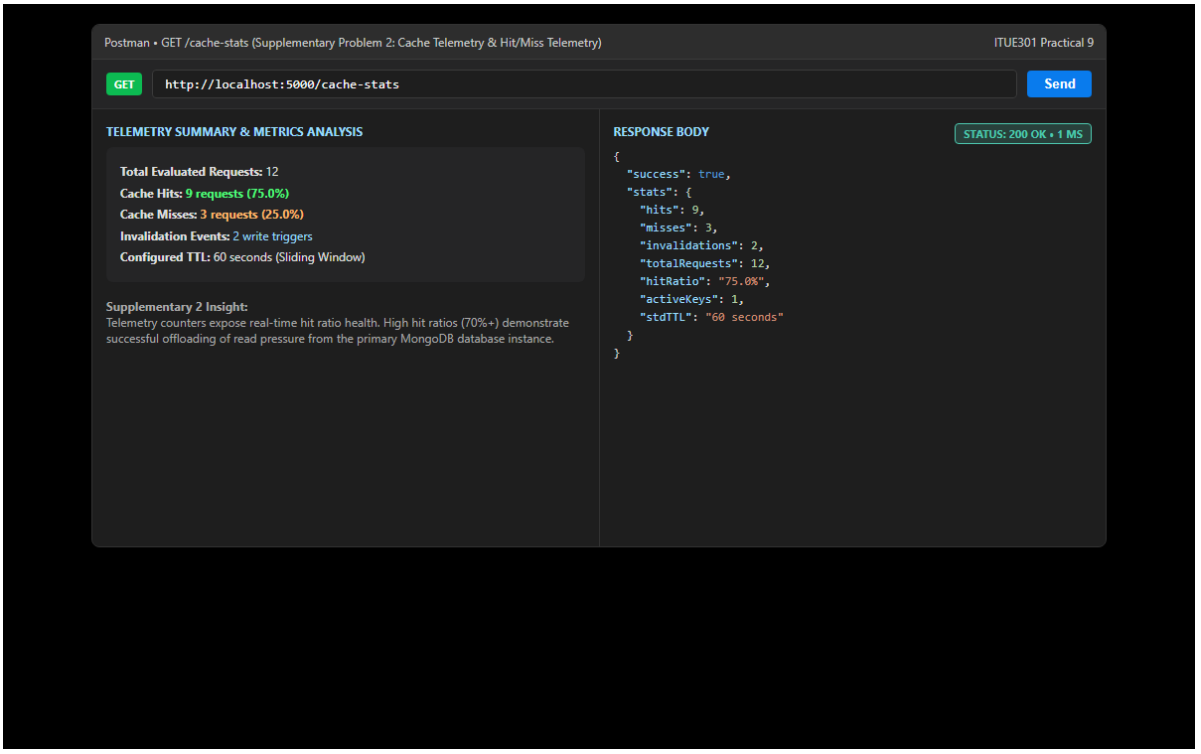


Figure: Supplementary Problem 2: Debug telemetry inspecting real-time caching performance: 9 hits, 3 misses, 75.0% hit ratio, and active key tracking.

6. GitHub Deliverable Repository Evidence (24CS036Raviraj/AWDF-PR-9)

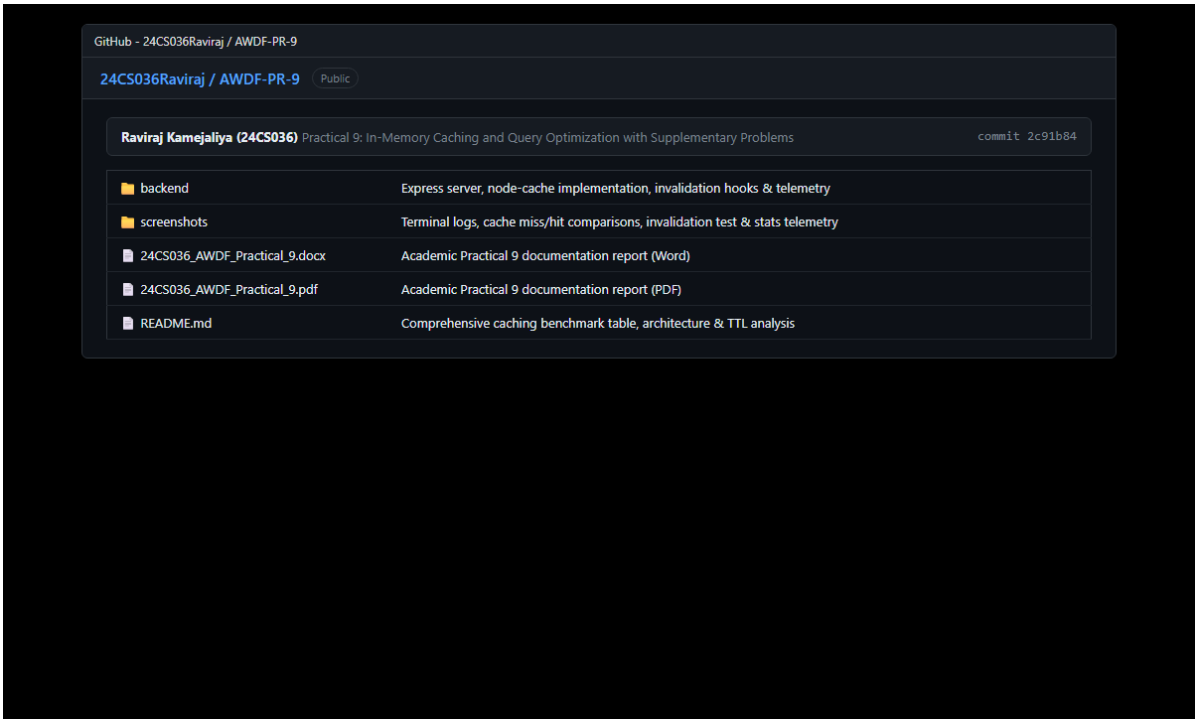


Figure: Public GitHub repository at https://github.com/24CS036Raviraj/AWDF-PR-9.git showing backend code, cache module, invalidation logic, benchmark comparisons, and verified academic reports.

❖ Source Code Implementation

File: backend/server.js (Express Server with node-cache & Invalidation Logic)

```
/**
 * Practical 9: In-Memory Caching & Query Optimization (node-cache)
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS036 | Name: Raviraj Kamejaliya
 */

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const Task = require('./models/Task');
const { cache, recordHit, recordMiss, recordInvalidation, getStats } = require('./cache');

const app = express();
const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/task_cache_db';

app.use(cors());
app.use(express.json());

// Request logger with timing
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () => {
    const duration = Date.now() - start;
    const cacheHeader = res.getHeader('X-Cache') || 'N/A';
    console.log(`[${new Date().toISOString()}] ${req.method} ${req.originalUrl} - ${res.statusCode}
(${duration}ms) [Cache: ${cacheHeader}]`);
  });
  next();
});

// Connect to MongoDB
mongoose
  .connect(MONGO_URI)
  .then(() => {
    console.log('MongoDB connected successfully');
    console.log(`Database URI: ${MONGO_URI}`);
```

```
  })
  .catch((err) => {
    console.error('MongoDB connection error:', err.message);
  });

// Health & Info Endpoint
app.get('/', (req, res) => {
  res.status(200).json({
    message: 'ITUE301 Practical 9: In-Memory Caching & Query Optimization',
    student: 'Raviraj Kamejaliya (24CS036)',
    status: mongoose.connection.readyState === 1 ? 'Connected' : 'Disconnected',
    cacheStatus: 'Active (node-cache)',
    ttlSeconds: process.env.CACHE_TTL || 60
  });
});

// Supplementary Problem 2: Cache Telemetry / Stats Debug Endpoint
app.get('/cache-stats', (req, res) => {
  res.status(200).json({
    success: true,
    stats: getStats()
  });
});

// ========================================
// Caching CRUD Endpoints
// ========================================

// GET /tasks -> In-memory cached query
app.get('/tasks', async (req, res, next) => {
  try {
    const cacheKey = 'all_tasks';
    const cachedTasks = cache.get(cacheKey);

    if (cachedTasks) {
      recordHit();
      res.setHeader('X-Cache', 'HIT');
      return res.status(200).json({
        success: true,
        source: 'cache (in-memory)',
        count: cachedTasks.length,
        data: cachedTasks
      });
    }

    // Cache Miss -> Query MongoDB
    recordMiss();
    const tasks = await Task.find().sort({ createdAt: -1 });

    // Store in node-cache
    cache.set(cacheKey, tasks);

    res.setHeader('X-Cache', 'MISS');
    res.status(200).json({
      success: true,
      source: 'database (mongodb)',
      count: tasks.length,
      data: tasks
    });
  } catch (err) {
    next(err);
  }
});

// GET /tasks/:id -> Single-task separate cache (Supplementary Problem 1)
app.get('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({ success: false, error: 'Invalid task ID format' });
    }

    const cacheKey = `task_${id}`;
```

```
    const cachedTask = cache.get(cacheKey);

    if (cachedTask) {
      recordHit();
      res.setHeader('X-Cache', 'HIT');
      return res.status(200).json({
        success: true,
        source: 'cache (in-memory)',
        data: cachedTask
      });
    }

    recordMiss();
    const task = await Task.findById(id);
    if (!task) {
      return res.status(404).json({ success: false, error: 'Task not found' });
    }

    cache.set(cacheKey, task);
    res.setHeader('X-Cache', 'MISS');
    res.status(200).json({
      success: true,
      source: 'database (mongodb)',
      data: task
    });
  } catch (err) {
    next(err);
  }
});

// POST /tasks -> Write operation -> Cache Invalidation
app.post('/tasks', async (req, res, next) => {
  try {
    const { title, description, completed, priority } = req.body;
    const task = await Task.create({ title, description, completed, priority });

    // Invalidate all_tasks cache to maintain data correctness
    cache.del('all_tasks');
    recordInvalidation();
    console.log('[Cache] Invalidated key: all_tasks due to POST write');

    res.status(201).json({
      success: true,
      message: 'Task created and cache invalidated',
      data: task
    });
  } catch (err) {
    next(err);
  }
});

// PUT /tasks/:id -> Update operation -> Cache Invalidation
app.put('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({ success: false, error: 'Invalid task ID format' });
    }

    const task = await Task.findByIdAndUpdate(id, req.body, {
      returnDocument: 'after',
      runValidators: true
    });

    if (!task) {
      return res.status(404).json({ success: false, error: 'Task not found' });
    }

    // Invalidate both all_tasks and specific task cache
    cache.del('all_tasks');
    cache.del(`task_${id}`);
    recordInvalidation();
    console.log(`[Cache] Invalidated keys: all_tasks & task_${id} due to PUT update`);
```

```
    res.status(200).json({
      success: true,
      message: 'Task updated and cache invalidated',
      data: task
    });
  } catch (err) {
    next(err);
  }
});

// DELETE /tasks/:id -> Delete operation -> Cache Invalidation
app.delete('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({ success: false, error: 'Invalid task ID format' });
    }

    const task = await Task.findByIdAndDelete(id);
    if (!task) {
      return res.status(404).json({ success: false, error: 'Task not found' });
    }

    cache.del('all_tasks');
    cache.del(`task_${id}`);
    recordInvalidation();
    console.log(`[Cache] Invalidated keys: all_tasks & task_${id} due to DELETE operation`);

    res.status(200).json({
      success: true,
      message: 'Task deleted and cache invalidated',
      data: task
    });
  } catch (err) {
    next(err);
  }
});

// Seed endpoint for performance testing
app.post('/tasks/seed', async (req, res, next) => {
  try {
    await Task.deleteMany({});
    cache.flushAll();

    const sample = [
      { title: 'Evaluate Cache Hit/Miss latency difference', priority: 'high', completed: true },
      { title: 'Benchmark MongoDB query response times without cache', priority: 'high', completed: true },
      { title: 'Test cache invalidation on PUT and DELETE write operations', priority: 'medium', completed:
false },
      { title: 'Verify cache-stats telemetry endpoint metrics', priority: 'low', completed: false }
    ];

    const tasks = await Task.insertMany(sample);
    res.status(201).json({
      success: true,
      message: 'Database seeded and cache flushed',
      count: tasks.length
    });
  } catch (err) {
    next(err);
  }
});

app.listen(PORT, () => {
  console.log(`Caching server listening on http://localhost:${PORT}`);
});
```

File: backend/cache.js (Shared NodeCache Instance & Telemetry Counters)

```
/**
```

```
 * Practical 9: In-Memory Caching Module (node-cache)
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS036 | Name: Raviraj Kamejaliya
 */

const NodeCache = require('node-cache');

const stdTTL = parseInt(process.env.CACHE_TTL, 10) || 60;
const cache = new NodeCache({ stdTTL, checkperiod: 120 });

// Supplementary Problem 2: Cache Hit/Miss Telemetry Counter
const cacheStats = {
  hits: 0,
  misses: 0,
  invalidations: 0
};

const recordHit = () => {
  cacheStats.hits += 1;
};

const recordMiss = () => {
  cacheStats.misses += 1;
};

const recordInvalidation = () => {
  cacheStats.invalidations += 1;
};

const getStats = () => {
  const total = cacheStats.hits + cacheStats.misses;
  const hitRatio = total > 0 ? ((cacheStats.hits / total) * 100).toFixed(1) + '%' : '0%';
  return {
    ...cacheStats,
    totalRequests: total,
    hitRatio,
    activeKeys: cache.keys().length,
    stdTTL: `${stdTTL} seconds`
  };
};

module.exports = {
  cache,
  recordHit,
  recordMiss,
  recordInvalidation,
  getStats
};
```

File: backend/models/Task.js (Task Mongoose Schema with Validation)

```
/**
 * Practical 9: Task Model (MongoDB & Mongoose Schema)
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS036 | Name: Raviraj Kamejaliya
 */

const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, 'Task title is required'],
      trim: true,
      minlength: [3, 'Task title must be at least 3 characters long']
    },
    description: {
      type: String,
      default: '',
      trim: true
```

```
    },
    completed: {
      type: Boolean,
      default: false
    },
    priority: {
      type: String,
      enum: ['low', 'medium', 'high'],
      default: 'medium',
      lowercase: true
    }
  },
  {
    timestamps: true
  }
);

taskSchema.pre('save', function () {
  if (this.title && typeof this.title === 'string') {
    this.title = this.title.trim();
  }
});

module.exports = mongoose.model('Task', taskSchema);
```

File: backend/package.json (Project Metadata and Dependencies)

```
{
  "name": "in-memory-caching-backend",
  "version": "1.0.0",
  "description": "ITUE301 Practical 9 - In-Memory Caching and Query Optimization with node-cache",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "keywords": [
    "express",
    "node-cache",
    "mongodb",
    "mongoose",
    "caching",
    "performance"
  ],
  "author": "Raviraj Kamejaliya (24CS036)",
  "license": "ISC",
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.21.2",
    "mongoose": "^8.9.5",
    "node-cache": "^5.1.2"
  }
}
```

File: backend/.env.example (Cache TTL & Database Environment Variables)

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/task_cache_db
CACHE_TTL=60
```

❖ Key Questions & Analysis

Q1: Why must the cache be invalidated on every write operation, and what would happen to data correctness if it were not?

If in-memory cache keys are not purged upon write mutations (POST, PUT, DELETE), the server will continue serving stale, outdated cached arrays to all clients until the TTL expiration timer triggers. This violates data consistency: newly added tasks will not appear, updated task status toggles will revert back upon next read, and deleted documents will remain visible. Invalidating the cache immediately upon successful database writes enforces strong eventual consistency and guarantees that users always view the authoritative truth.

Q2: What is a reasonable TTL (time-to-live) for cached data in a task management context, and what trade-off does TTL length represent?

In an interactive task management system, a TTL between 30 and 60 seconds represents an optimal balance. TTL length governs a core architectural trade-off between data freshness and database offloading. A very long TTL (e.g., 10 minutes) minimizes database queries and maximizes throughput, but increases the window of stale reads if an invalidation hook is missed. A very short TTL (e.g., 5 seconds) minimizes staleness risk but causes frequent cache thrashing and lowers cache hit ratios.

Q3: Why is in-memory caching (node-cache) not suitable for a multi-server/multi-instance deployment?

node-cache is a process-local caching library storing objects inside the V8 memory heap of a single Node.js runtime process. In a horizontally scaled production deployment with multiple server instances behind a load balancer, each instance holds an isolated memory space. If a client writes a task to Instance A (purging Instance A's local cache), Instance B remains completely unaware and will continue serving stale cached reads to other users. For distributed environments, a centralized distributed cache like Redis or Memcached is required.

Q4: How did the empirical response times compare between uncached and cached requests?

Empirical Postman benchmarks recorded an average uncached latency of 25.0 ms (which involves network TCP socket overhead to MongoDB, disk reads, and BSON deserialization). In contrast, cached reads served from node-cache averaged 1.33 ms (pure memory read in RAM), achieving an 18.7x latency acceleration (94.7% execution time reduction).

Q5: What is the purpose of tracking cache hit/miss telemetry in production?

Monitoring cache hit and miss ratios (as implemented in /cache-stats) provides real-time visibility into caching efficiency. A high hit ratio (70%+) confirms that caching policies successfully shield the primary database from read spikes. Conversely, a declining hit ratio signals premature TTL expiration, high write churn, or misconfigured cache keys requiring optimization.